

DataFrame 그룹연산 · 결측치 · 병합 · 변환

groupby / dropna / merge / apply 외 19개 명령어 | 광명코딩학원

> 여러 행을 기준으로 묶어 통계 내기(groupby), 결측치/중복 처리(dropna, fillna, duplicated), 여러 DataFrame을 합치기(merge, concat), 값을 변환하기(apply, map, astype)는 데이터 분석의 핵심 작업입니다.

PART 1. 그룹 연산 (groupby / mean / agg / pivot_table)

1. groupby — 그룹별로 묶기

특정 컬럼의 값을 기준으로 데이터를 그룹으로 묶음. 그 자체로는 결과가 안 보이고, mean(), sum() 등 집계 함수를 붙여야 결과가 나옴.

```
from pandas import DataFrame

data = {
    "지점": ["강남", "강남", "홍대", "홍대", "잠실", "잠실"],
    "카테고리": ["음료", "디저트", "음료", "디저트", "음료", "디저트"],
    "매출": [50000, 30000, 42000, 28000, 61000, 35000]
}
df = DataFrame(data)

group = df.groupby("지점")
print(group)
# <pandas.core.groupby.generic.DataFrameGroupBy object ...>
# ← 그룹 객체만 생성, 집계 함수 필요
```

2. groupby + mean — 그룹별 평균

```
print(df.groupby("지점")["매출"].mean())
# 지점
# 강남      40000.0
# 잠실      48000.0
# 홍대      35000.0
# Name: 매출, dtype: float64
```

```
# 여러 컬럼 기준 그룹 (지점 + 카테고리)
print(df.groupby(["지점", "카테고리"])["매출"].mean())
# 지점  카테고리
# 강남  디저트    30000.0
#      음료     50000.0
# 잠실  디저트    35000.0
#      음료     61000.0
# 홍대  디저트    28000.0
#      음료     42000.0
```

함수	의미
.mean()	평균
.sum()	합계
.count()	개수
.max() / .min()	최댓값/최솟값

3. agg — 여러 집계함수 한꺼번에

groupby 뒤에 mean() 하나만 붙이는 대신, agg()로 여러 통계를 동시에 뽑을 수 있음.

```
print(df.groupby("지점")["매출"].agg(["mean", "sum", "count"]))
#      mean      sum  count
# 지점
# 강남   40000.0   80000     2
# 잠실   48000.0   96000     2
# 홍대   35000.0   70000     2
```

```
# 컬럼별로 다른 집계함수 적용
print(df.groupby("지점").agg({"매출": ["mean", "max"]})))
```

4. pivot_table — 엑셀 피벗테이블처럼 펼쳐보기

groupby와 비슷한 결과를 만들지만, 그룹 기준을 행/열로 펼쳐서 표 형태로 보여줌. 실무에서 groupby보다 더 자주 쓰임.

```
print(df.pivot_table(index="지점", columns="카테고리", values="매출"))
# 카테고리  디저트    음료
# 지점
# 강남      30000.0  50000.0
# 잠실      35000.0  61000.0
# 홍대      28000.0  42000.0
```

groupby는 결과가 세로로 길게 쌓이고(Series/MultiIndex), pivot_table은 행×열 표로 펼쳐짐 — 보기 편한 쪽을 선택.

PART 2. 결측치 · 중복 처리 (dropna / fillna / duplicated / drop_duplicates)

5. dropna — 결측치(NaN) 제거

```
import numpy as np

data = {
    "이름": ["철수", "영희", "민수", "지수"],
    "점수": [90, np.nan, 85, np.nan]
}
df = DataFrame(data)
print(df)
#   이름   점수
# 0  철수  90.0
# 1  영희   NaN
# 2  민수  85.0
# 3  지수   NaN

df2 = df.dropna()      # NaN이 있는 행 전체 삭제 (원본 유지)
print(df2)
#   이름   점수
# 0  철수  90.0
# 2  민수  85.0

df.dropna(inplace=True) # 원본에 바로 반영
```

주의: dropna()는 기본적으로 행(axis=0) 전체를 삭제. axis=1을 주면 NaN이 있는 열을 삭제.

6. fillna — 결측치를 다른 값으로 채우기

삭제하지 않고 평균값, 0, 특정 문자열 등으로 채워서 데이터를 유지.

```

df = DataFrame({"점수": [90, np.nan, 85, np.nan]})

print(df.fillna(0))
#   점수
# 0  90.0
# 1   0.0
# 2  85.0
# 3   0.0

# 평균값으로 채우기 (더 많이 씀)
print(df.fillna(df["점수"].mean()))
#   점수
# 0  90.0
# 1  87.5
# 2  85.0
# 3  87.5

```

7. duplicated / drop_duplicates — 중복 행 확인/제거

```

df = DataFrame({
    "이름": ["철수", "영희", "철수", "민수", "철수"],
    "점수": [90, 85, 90, 78, 95]
})

print(df.duplicated())
# 0    False
# 1    False
# 2     True   ← 0번과 완전히 동일한 행
# 3    False
# 4    False
# dtype: bool

print(df.drop_duplicates()) # 기본: 모든 컬럼이 같아야 중복, 첫 번째만 남김
#   이름  점수
# 0  철수   90
# 1  영희   85
# 3  민수   78
# 4  철수   95   ← 점수가 다르므로 중복 아님

```

```

# subset: 특정 컬럼만 기준으로 중복 판단
print(df.drop_duplicates(subset="이름")) # 이름만 같으면 중복으로 봄
#   이름  점수
# 0  철수   90
# 1  영희   85
# 3  민수   78

# keep: 어느 것을 남길지 (기본 'first', 'last'도 가능)
print(df.drop_duplicates(subset="이름", keep="last"))
#   이름  점수
# 1  영희   85
# 3  민수   78
# 4  철수   95 ← 철수 중 마지막 행 남김

```

subset을 안 주면 모든 컬럼 값이 같아야 중복. keep="last"는 중복 중 마지막 행을 남김.

PART 3. 병합 · 연결 (merge / concat)

8. merge — 키(기준 컬럼)로 두 DataFrame 합치기

```
df1 = DataFrame({
    "고객ID": [1, 2, 3, 4],
    "이름": ["철수", "영희", "민수", "지수"]
})

df2 = DataFrame({
    "고객ID": [1, 2, 3, 5],
    "구매금액": [10000, 20000, 15000, 30000]
})
```

inner (기본값) — 양쪽에 공통으로 있는 고객ID만

```
print(pd.merge(df1, df2, on="고객ID", how="inner"))
#   고객ID  이름  구매금액
# 0     1   철수   10000
# 1     2   영희   20000
# 2     3   민수   15000
# ← 고객ID 4(df1에만), 5(df2에만)는 빠짐
```

outer — 양쪽 전체, 없는 쪽은 NaN

```
print(pd.merge(df1, df2, on="고객ID", how="outer"))
#   고객ID  이름  구매금액
# 0     1   철수   10000.0
# 1     2   영희   20000.0
# 2     3   민수   15000.0
# 3     4   지수      NaN
# 4     5   NaN   30000.0
```

how	의미
inner	양쪽 다 있는 것만 (교집합)
outer	양쪽 전체 (합집합), 없는 쪽 NaN
left	왼쪽(df1) 기준 전체
right	오른쪽(df2) 기준 전체

9. concat — 단순 연결 (위/아래, 좌/우로 이어붙이기)

merge는 "공통 키 기준 결합", concat은 "그냥 이어붙이기". 키가 없어도 됨.

```

df1 = DataFrame({"이름": ["철수", "영희"], "점수": [90, 85]})
df2 = DataFrame({"이름": ["민수", "지수"], "점수": [78, 92]})

# 위/아래로 연결 (기본, axis=0)
print(pd.concat([df1, df2]))
#   이름  점수
# 0 철수   90
# 1 영희   85
# 0 민수   78  ← 인덱스가 0,1로 중복됨
# 1 지수   92

print(pd.concat([df1, df2], ignore_index=True)) # 인덱스 재정렬
#   이름  점수
# 0 철수   90
# 1 영희   85
# 2 민수   78
# 3 지수   92

```

```

# 좌/우로 연결 (axis=1)
print(pd.concat([df1, df2], axis=1))

```

merge vs concat — merge는 "고객ID 같은 키로 컬럼을 옆으로 붙이기", concat은 "그냥 행이나 열을 이어붙이기"

PART 4. 함수 적용 · 값 변환 (apply / map / tolist / replace)

10. apply — 각 값/행에 함수 적용

```
df = DataFrame({
    "이름": ["철수", "영희", "민수"],
    "점수": [85, 92, 78]
})

def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    else:
        return "C"

df["등급"] = df["점수"].apply(grade)
print(df)
#   이름  점수  등급
# 0  철수   85    B
# 1  영희   92    A
# 2  민수   78    C
```

```
df["점수2배"] = df["점수"].apply(lambda x: x * 2)  # lambda로 한 줄 처리
```

11. map — Series 값을 딕셔너리로 매핑

apply와 비슷하지만 Series 전용. 딕셔너리로 값을 1:1 치환할 때 더 간결.

```
df["등급설명"] = df["등급"].map({"A": "우수", "B": "보통", "C": "미흡"})
print(df)
#   이름  점수  등급  등급설명
# 0  철수   85    B    보통
# 1  영희   92    A    우수
# 2  민수   78    C    미흡
```

12. tolist — Series를 파이썬 리스트로 변환

Series나 Index를 파이썬 리스트로 변환. 반복문(for)이나 다른 라이브러리에 값만 넘길 때 자주 사용.

```
names = df["이름"].tolist()
print(names)
# ['철수', '영희', '민수']
```

```
scores = df["점수"].tolist()
print(scores)
# [85, 92, 78]
```

```
# for문에서 활용 예
for name in df["이름"].tolist():
    print(f"{name}님 안녕하세요")
# 철수님 안녕하세요
# 영희님 안녕하세요
# 민수님 안녕하세요
```

df["컬럼"]만 출력하면 Series(인덱스 포함)지만, .tolist()를 붙이면 인덱스 없는 순수 값 리스트만 남음.

13. replace — 특정 값을 다른 값으로 일괄 치환

```
df["등급"] = df["등급"].replace("C", "D") # 값 자체를 바꿈
print(df)
```

#	이름	점수	등급	등급설명
# 0	철수	85	B	보통
# 1	영희	92	A	우수
# 2	민수	78	D	미흡

map은 "딕셔너리 전체 매핑"에, replace는 "특정 값 몇 개만 바꾸기"에 더 적합.

PART 5. 정렬 · 타입 · 컬럼 관리

14. sort_values — 값 기준 정렬

```
print(df.sort_values("점수", ascending=False)) # 점수 내림차순
#   이름  점수  등급  등급설명
# 1  영희   92    A    우수
# 0  철수   85    B    보통
# 2  민수   78    D    미흡
```

15. astype — 데이터 타입 변환

문자열을 숫자로, 숫자를 문자열로 등. 실무에서 CSV 불러온 뒤 타입이 안 맞을 때 자주 사용.

```
df = DataFrame({"점수": ["90", "85", "78"]}) # 문자열로 저장된 점수
print(df.dtypes)
# 점수    object

df["점수"] = df["점수"].astype(int)
print(df.dtypes)
# 점수    int64
```

16. rename — 컬럼명 변경

```
df = df.rename(columns={"점수": "시험점수"})
print(df.columns)
# Index(['시험점수'], dtype='object')
```

17. reset_index — 인덱스 초기화

groupby나 필터링 후 인덱스가 0,1,2... 순서가 아니게 꼬였을 때 0부터 다시 매김.

```
df = DataFrame({"이름": ["철수", "영희", "민수"]}, index=[5, 8, 2])
print(df)
#   이름
# 5  철수
# 8  영희
# 2  민수

print(df.reset_index(drop=True)) # drop=True: 기존 인덱스를 버림
#   이름
# 0  철수
# 1  영희
# 2  민수
```

drop=True가 없으면 기존 인덱스가 새로운 컬럼으로 남음.

18. value_counts — 값별 개수 세기 (컬럼 단위)

```
df = DataFrame({"등급": ["A", "B", "A", "C", "B", "A"]})
print(df["등급"].value_counts())
# A    3
# B    2
# C    1
# Name: count, dtype: int64
```

[완료] groupby부터 value_counts까지 19개 명령어를 모두 학습했습니다. 그룹 통계, 결측치/중복 처리, 병합, 값 변환까지 — 실무 데이터 분석의 기초가 완성되었습니다!